

Roteiro de apresentação do código - NightBull

Este material organiza os principais pontos técnicos para apresentar o aplicativo de carteira de investimentos. A ideia é mostrar onde cada parte está no código e como os recursos principais funcionam.

1. Visão geral do projeto

O projeto está organizado dentro da pasta src, separando telas, banco de dados, serviços, navegação, componentes, tema e utilitários. Essa divisão ajuda a explicar o app por responsabilidade, sem misturar lógica de interface, persistência e integração externa.

- src/screens: telas principais do app.
- src/database: criação do banco SQLite e consultas.
- src/services: cotações, notícias e mocks controlados.
- src/navigation: configuração das abas inferiores.
- src/components: componentes reutilizáveis.
- src/theme e src/utlis: paleta, tipografia e funções auxiliares.

2. Arquivo principal do aplicativo

O arquivo App.js carrega fontes, inicializa o banco local e decide qual fluxo será exibido. Se não houver usuário autenticado, a tela de login é mostrada. Após o login, o app carrega a navegação principal.

```
initializeDatabase()
```

Essa chamada prepara as tabelas e garante que o SQLite esteja pronto antes de liberar o uso do aplicativo.

3. Navegação por abas

A navegação fica em src/navigation/AppNavigator.js e usa @react-navigation/bottom-tabs. As abas representam o fluxo do usuário: Carteira, Operar, Relatórios, Notícias e Sobre.

O usuário logado é repassado para as telas que precisam consultar dados específicos da conta.

```
<HomeScreen currentUser={currentUser} />  
<TradeScreen currentUser={currentUser} />  
<ReportsScreen currentUser={currentUser} />  
<NewsScreen currentUser={currentUser} />
```

4. Banco de dados local com SQLite

A persistência local está em src/database/database.js. O app usa expo-sqlite para criar e consultar as tabelas users e transactions.

- users: armazena id, nome, e-mail, senha e data de criação.
- transactions: armazena user_id, tipo da operação, ticker, quantidade, preço e data.

O campo user_id é importante porque separa a carteira de cada usuário. Assim, um login não mistura transações de outra conta.

5. Cadastro e login

A tela de autenticação está em `src/screens/AuthScreen.js`. Ela chama funções do banco para criar usuário ou validar login.

```
createUser({ name, email, password })
```

```
authenticateUser({ email, password })
```

Quando o login é aceito, o usuário autenticado é enviado ao `App.js`, que passa a exibir a navegação principal.

6. Registro de ações, FIIs e ETFs

A tela de operação está em `src/screens/TradeScreen.js`. Nela, o usuário digita parte do ticker e recebe uma lista rolável de ativos compatíveis.

Depois que o usuário seleciona um ativo, a cotação é carregada automaticamente. O botão manual de buscar cotação foi removido porque a seleção do ativo já dispara a consulta.

- O usuário escolhe o ativo na lista.
- A cotação atual é carregada como sugestão.
- O preço pago pode ser alterado manualmente.
- A operação é salva como BUY ou SELL no SQLite.

7. Como a API da Brapi é chamada

A integração com a API fica em `src/services/marketApi.js`, principalmente na função `fetchQuote`.

```
const response = await fetch(`${BRAPI_BASE_URL}/${cleanTicker}`);
```

A URL usada segue o formato `https://brapi.dev/api/quote/{ticker}`. Depois da resposta, o app lê o JSON e extrai o primeiro resultado retornado pela API.

```
const payload = await response.json();
```

```
const result = payload?.results?.[0];
```

O retorno é padronizado com ticker, preço, variação, nome do ativo e a origem da cotação. Se a Brapi falhar, alguns ativos conhecidos usam fallback local para não quebrar a apresentação.

8. Como a transação é salva

A gravação acontece pela função `addTransaction`, em `src/database/database.js`. Ela recebe `userId`, `tipo`, `ticker`, `quantidade`, `preço` e `data`.

```
await addTransaction({ userId, type, ticker, quantity, price, date });
```

O preço salvo é o preço pago informado pelo usuário, não necessariamente a cotação atual. Isso é importante porque uma compra real pode ter sido feita em outro momento.

9. Como a carteira é calculada

A função `calculatePositions` percorre as transações do usuário e agrupa tudo por ticker.

Nas compras, aumenta quantidade e custo total. Nas vendas, reduz a quantidade e ajusta o custo usando o preço médio.

- Calcula quantidade atual.

- Calcula custo total remanescente.
- Calcula preço médio.
- Guarda compras e vendas acumuladas.
- Calcula resultado realizado em vendas.

10. Tela Carteira

A tela Carteira fica em `src/screens/HomeScreen.js`. Ela lê as posições atuais, busca cotações, calcula patrimônio total e mostra os cards dos ativos.

```
getCurrentPositions(currentUser.id)
fetchQuotes(...)
```

A mesma tela também mostra histórico de transações e permite abrir o detalhe de um ativo tocando no card.

11. Detalhe do ativo

O detalhe do ativo está dentro da `HomeScreen`, no componente `AssetDetail`. Ele mostra preço médio, preço atual, custo remanescente, resultado não realizado, compras, vendas e histórico específico daquele ticker.

12. Relatórios

A tela de relatórios está em `src/screens/ReportsScreen.js`. Ela lê as posições, atualiza os preços e monta os dados visuais.

A classificação por tipo fica em `src/utils/assetClassifier.js`, separando ações, FIIs, ETFs e outros.

- Gráficos de pizza são desenhados com `react-native-svg`.
- O gráfico de custo vs. valor atual foi feito com Views customizadas.
- Os cards mostram total investido, valor atual, resultado e quantidade de ativos.

13. Notícias

A tela de notícias fica em `src/screens/NewsScreen.js`. O serviço de notícias fica em `src/services/marketApi.js`.

Existem duas listas mockadas: notícias gerais de mercado e notícias relacionadas aos ativos da carteira. O filtro da tela alterna entre Gerais e Meus ativos.

```
newsScope === "portfolio" ? portfolioNews : generalNews
```

14. Como abre o site ao clicar na notícia

Cada card de notícia é um `Pressable`. Ao tocar no card, a função `openSource` usa `Linking.openURL` para abrir a fonte externa.

```
import { Linking } from "react-native";
await Linking.openURL(item.url);
```

Dessa forma, o app mantém um card estilizado internamente, mas leva o usuário para a fonte original quando ele quer ler mais.

15. Tema visual

As cores e fontes ficam centralizadas em `src/theme/colors.js` e `src/theme/typography.js`. Isso evita repetir valores espalhados pelo app e ajuda a manter o dark theme consistente.

16. Componentes reutilizáveis

A pasta `src/components` concentra elementos usados em várias telas, como `Header`, `ScreenContainer`, `SectionLabel`, `EmptyState` e `AssetCard`.

- `ScreenContainer` padroniza fundo, scroll e espaçamento.
- `Header` padroniza título e marca visual.
- `AssetCard` mostra ticker, quantidade, preço médio, preço atual e variação.

17. Sequência sugerida para apresentar o código

- Começar pelo `App.js` para explicar inicialização, fontes, banco e autenticação.
- Abrir `AppNavigator.js` para mostrar a navegação por abas.
- Abrir `database.js` para explicar `users`, `transactions` e `user_id`.
- Abrir `TradeScreen.js` para mostrar autocomplete, cotação e registro da operação.
- Abrir `marketApi.js` para explicar `Brapi`, `fallback` e notícias.
- Abrir `HomeScreen.js` para explicar agrupamento por ticker e cálculo da carteira.
- Abrir `ReportsScreen.js` para mostrar gráficos e classificação por tipo.
- Abrir `NewsScreen.js` para mostrar filtros e `Linking.openURL`.